

.NET 10 · C# CONSOLE APP

Item Directions

From a tree drawn in text to step-by-step directions

Matias · Interview presentation

THE TASK

Pick an item with one key press, get directions to it

Input: Data.small.txt (excerpt)

```
+ Walk to the end of the hall.  
|+ Turn left.  
|  |+ Go through the first door on the right.  
|    |+ Open the cabinet on the left.  
|      | Item: Cookies  
|      |+ Open the refridgerator.  
|        | Item: Milk  
|+ Turn right.  
|  |+ Go through the door at the end of the hall.  
|    |+ Look on top of the desk.  
|      | Item: Mobile Phone
```

Output: the console app

```
Available items:  
[1] - Pencils  
[2] - Mobile Phone  
[3] - Milk  
[4] - Cookies  
[5] - Coffee Mug  
What item would you like to search for?  
2  
Walk to the end of the hall.  
Turn right.  
Go through the door at the end of the hall.  
Look on top of the desk.
```

KEY INSIGHT

The data file is an n-ary tree drawn in text

+ text

Direction – an inner node

Item: name

Item – a leaf, unique in the file



Connector to the parent (└─ = last child)

| and spaces

Indentation only, 3 characters per level

Line	Connector at	Depth
+ Walk to the end...	none	0
└─+ Turn left.	0	1
└─+ Go through...	3	2
└─+ Open the...	6	3
└─ Item: ...	9	4

depth = connectorIndex / 3 + 1

ARCHITECTURE

Small classes, one job each

Program.cs

Picks the data source; reports any error as FAILURE:

DataFileSelector

Uses Data.txt if present, otherwise lets you pick a file from TestData

DataFileParser

Validates every line and builds the tree

Node

A direction or an item: Text, Children, Parent, GetDirections()

ItemMenu

Reverse-sorted list, one key press, prints the path, repeats

SelectionKeys

Maps list positions to keys 1–9, then a–z

HOW PARSING WORKS

Two passes: read the lines, then build the tree

PASS 1 · PARSELINE

Each line becomes (depth, isItem, text). Nothing is guessed:

- ✓ connector sits on the 3-character grid
- ✓ only | and spaces in front of it
- ✓ connector is |— or —|
- ✓ line starts with "+ " or "Item: "
- ✓ no blank lines, no empty text

PASS 2 · PARSECHILDREN (RECURSIVE)

Attaches lines at the current depth; every direction recurses one level deeper.

```
if (line.Depth < depth) return; // closed
if (line.Depth > depth) throw ...; // skip
parent.AddChild(node);
if (!line.IsItem)
    ParseChildren(node, depth + 1);
```

Path to an item = walk up through Parent to the root and reverse. An iterative version with an explicit Stack<Node> is in git history.

FAIL FAST

A broken file stops with a clear message

FAILURE: Could not load the data file. Line 3: Blank lines are not allowed.

Problem in the file	Message
File or folder is missing	Data file 'Data.txt' was not found.
Blank line	Line 3: Blank lines are not allowed.
Skipped level / item with children	Line 2: Unexpected indentation.
Same item twice	Line 4: Item 'Milk' is defined more than once.
Connector off the grid	Line 2: Connector is not aligned to the indentation grid.

LIVE DEMO

Let's run it

1 Pick a data file

2 Choose an item

3 Letters for more than 9 items

4 A broken file → FAILURE:

Questions?